Newton-Raphson Method

Problem 1.2.16 — EC 2007

EE25BTECH11016 - CH. Taraka Abhinav

Problem 1.2.16 – EC 2007

The equation

$$\mathbf{f(x) = x^2 + x^4 - 4x - 4 = 0}$$

is to be solved using Newton-Raphson. If $x = 2$ is the initial approximation, then the next approximation is:

a. $\frac{2}{3}$

b. $\frac{4}{3}$

c. 1

d. $\frac{3}{2}$

The Newton-Raphson Formula

The Newton-Raphson iteration formula is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

For **one** iteration from $x_0 = 2$:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 2 - \frac{f(2)}{f'(2)}$$

We need two things: $f(2)$ and $f'(2)$

Setting Up – Function & Derivative

The function:

$$f(x) = x^4 + x^2 - 4x - 4$$

Differentiating term by term:

$$f'(x) = 4x^3 + 2x - 4$$

Term	**Derivative**
x^4	$4x^3$
x^2	$2x$
$-4x$	-4
-4	0

Evaluating $f(2)$

$$f(x) = x^4 + x^2 - 4x - 4$$

Substituting $x = 2$:

$$f(2) = (2)^4 + (2)^2 - 4(2) - 4$$

$$f(2) = 16 + 4 - 8 - 4$$

$$f(2) = 8$$

Evaluating $f'(2)$

$$f'(x) = 4x^3 + 2x - 4$$

Substituting $x = 2$:

$$f'(2) = 4(2)^3 + 2(2) - 4$$

$$f'(2) = 4 \times 8 + 4 - 4$$

$$f'(2) = 32 + 4 - 4$$

$$f'(2) = 32$$

Applying the Newton-Raphson Formula

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 2 - \frac{f(2)}{f'(2)}$$

Substituting the computed values:

$$x_1 = 2 - \frac{8}{32}$$

$$x_1 = 2 - \frac{1}{4}$$

$$x_1 = \frac{8}{4} - \frac{1}{4}$$

$$\Rightarrow \boxed{x_1 = \frac{7}{4}}$$

Derivation via Taylor Series

Let x^* be the exact root, meaning $f(x^*) = 0$.
Expanding $f(x)$ using the Taylor series around our guess x_n:

$$f(x^*) = f(x_n) + (x^* - x_n)f'(x_n) + \frac{(x^* - x_n)^2}{2!}f''(\xi) = 0$$

For a small interval, we truncate the higher-order terms ($x^* \approx x_{n+1}$):

$$f(x_n) + (x_{n+1} - x_n)f'(x_n) \approx 0$$

Rearranging algebraically to solve for the next iteration x_{n+1}:

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}}$$

Error Analysis & Asymptotic Constant

Let the error at step n be $e_n = x_n - x^*$.
Using the exact Taylor expansion, we find the relation:

$$e_{n+1} \approx \frac{f''(x^*)}{2f'(x^*)} e_n^2$$

For a simple root ($f'(x^*) \neq 0$), the sequence exhibits **Quadratic Convergence** ($p = 2$). The **Asymptotic Error Constant (M)** is defined as:

$$\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^2} = M = \left| \frac{f''(x^*)}{2f'(x^*)} \right|$$

Practical Meaning: The number of correct decimal places essentially **doubles** with every single iteration once x_n is close to x^*.

Conditions for Guaranteed Convergence

Fourier's Sufficient Condition: Convergence is mathematically guaranteed if the initial guess x_0 satisfies:

$$\left|\frac{f(x)f''(x)}{[f'(x)]^2}\right| < 1 \quad \text{for all } x \text{ in the interval.}$$

Verifying our problem at $x_0 = 2$:

$$f(2) = 8, \quad f'(2) = 32$$

$$f''(x) = 12x^2 + 2 \implies f''(2) = 12(2)^2 + 2 = 50$$

$$\left|\frac{f(2)f''(2)}{[f'(2)]^2}\right| = \frac{8 \times 50}{(32)^2} = \frac{400}{1024} \approx 0.39 < 1$$

Condition strongly satisfied! Convergence from $x_0 = 2$ is absolute.

Advanced Extensions: Multidimensional NR

In M.Tech/Advanced applications (e.g., Load Flow Analysis in Power Systems), we solve systems of non-linear equations $\mathbf{F}(\mathbf{x}) = 0$.

The scalar derivative $f'(x)$ is replaced by the **Jacobian Matrix** $\mathbf{J}(\mathbf{x})$, containing partial derivatives:

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

The iteration formula becomes a matrix operation:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n)$$

Handling Pathologies: Multiple Roots

If a root has a multiplicity $m > 1$ (i.e., $f(x^*) = 0$ and $f'(x^*) = 0$), standard Newton-Raphson degrades to **Linear Convergence**.

To restore quadratic convergence, we use the **Modified Newton-Raphson Method**:

$$x_{n+1} = x_n - m\frac{f(x_n)}{f'(x_n)}$$

Alternatively, applying standard NR to the function $u(x) = \frac{f(x)}{f'(x)}$

always yields quadratic convergence, regardless of root multiplicity.

Global Dynamics: Basins of Attraction

Newton's method is highly sensitive to the initial guess x_0.

When extended to the **Complex Plane** ($z \in \mathbb{C}$), the roots act as "attractors". The set of all initial guesses that converge to a specific root is called its **Basin of Attraction**.

For our function $f(z) = z^4 + z^2 - 4z - 4$, coloring the complex plane based on which of the 4 roots an initial guess converges to generates a **Newton Fractal**. The boundaries between these basins are infinitely complex and chaotic.

Convergence of Iterations

Starting with $x_0 = 2$:

$$x_1 = 1.75$$

$$x_2 \approx 1.68115672$$

$$x_3 \approx 1.67628062$$

$$x_4 \approx 1.67625719$$

After several iterations:

$$x_n \longrightarrow 1.67$$

Hence, the root of the equation is approximately:

$$\boxed{1.67}$$

Final Verification via Limit

From Newton-Raphson iterations:

$$x_n \longrightarrow 1.67$$

Let the exact root be:

$$x^* = \lim_{n \to \infty} x_n$$

Since Newton-Raphson converges:

$$f(x^*) = 0$$

Final Verification via Limit

Numerical verification:

$$f(1.67) = (1.67)^4 + (1.67)^2 - 4(1.67) - 4$$

$$= 7.78 + 2.79 - 6.68 - 4$$

$$= 10.57 - 10.68$$

$$f(1.67) = -0.112 \neq 0$$

for x = 1.673:

$$f(1.673) \approx -0.01$$

$$f(1.6735) \approx 0$$

Final Verification via Limit

Numerical verification: Hence:

$$\lim_{n\to\infty} f(x_n) = 0$$

$$\boxed{x^* \approx 1.673}$$

$$Error \to 0 \text{ as iterations} \to \infty$$

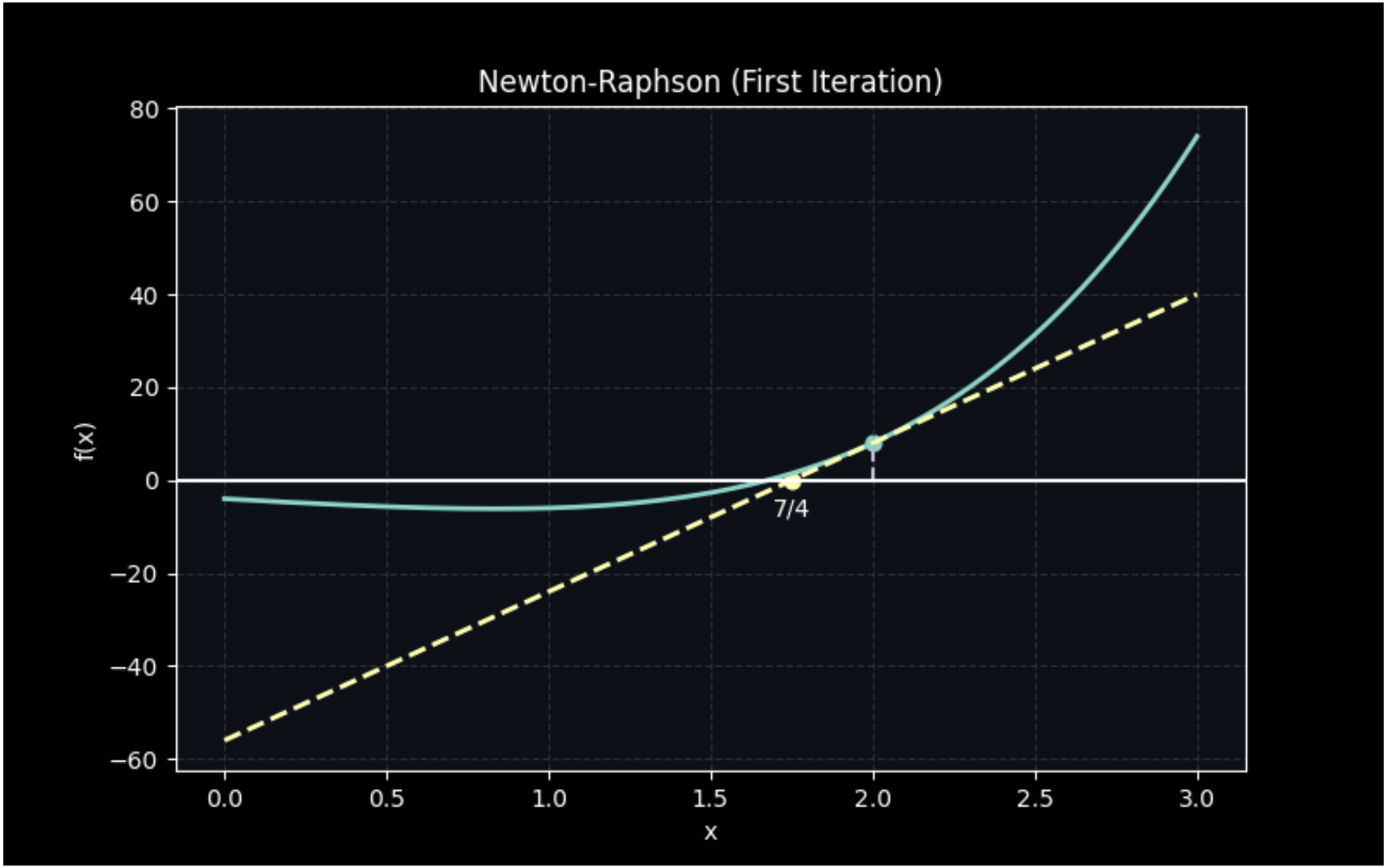
Python Code: Newton Plot

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
   return x**4 + x**2 - 4*x - 4

def df(x):
   return 4*x**3 - 2*x - 4

x0 = 2
x1 = x0 - f(x0)/df(x0)

plt.style.use('dark_background')

fig1, ax1 = plt.subplots(figsize=(8,5))

x = np.linspace(0, 3, 400)
ax1.plot(x, f(x))

tangent = f(x0) + df(x0)*(x - x0)
ax1.plot(x, tangent, '--')
```

Python Code: Newton Plot

```
ax1.scatter(x0, f(x0))
ax1.scatter(x1, 0)

ax1.plot([x0, x0], [0, f(x0)], '--')

ax1.annotate("7/4", (x1, 0),
 textcoords="offset points",
 xytext=(0, -15), ha='center')

plt.show()
```

Newton-Raphson First Iteration

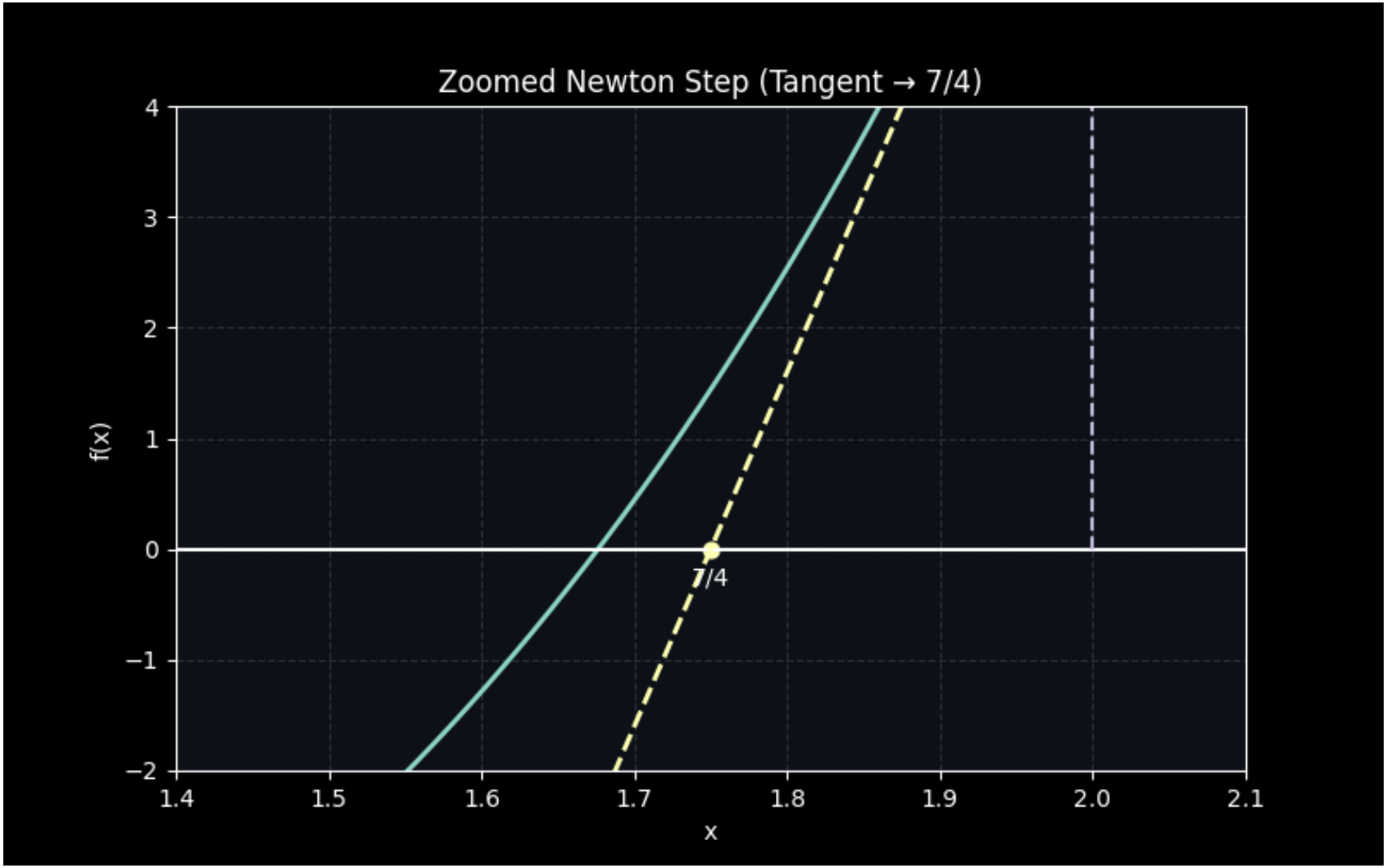


$$\textit{Tangent at } x_0 = 2 \textit{ intersects x-axis at } x_1 = \frac{7}{4}$$

Python Code: Zoomed Plot

```
fig2, ax2 = plt.subplots(figsize=(8,5))

x_zoom = np.linspace(1.4, 2.1, 400)

ax2.plot(x_zoom, f(x_zoom))
ax2.axhline(0)

ax2.scatter(x0, f(x0))
ax2.scatter(x1, 0)

tangent_zoom = f(x0) + df(x0)*(x_zoom - x0)
ax2.plot(x_zoom, tangent_zoom, '--')

ax2.plot([x0, x0], [0, f(x0)], '--')

ax2.annotate("7/4", (x1, 0),
 textcoords="offset points",
 xytext=(0, -15), ha='center')

plt.show()
```

Zoomed Newton Step

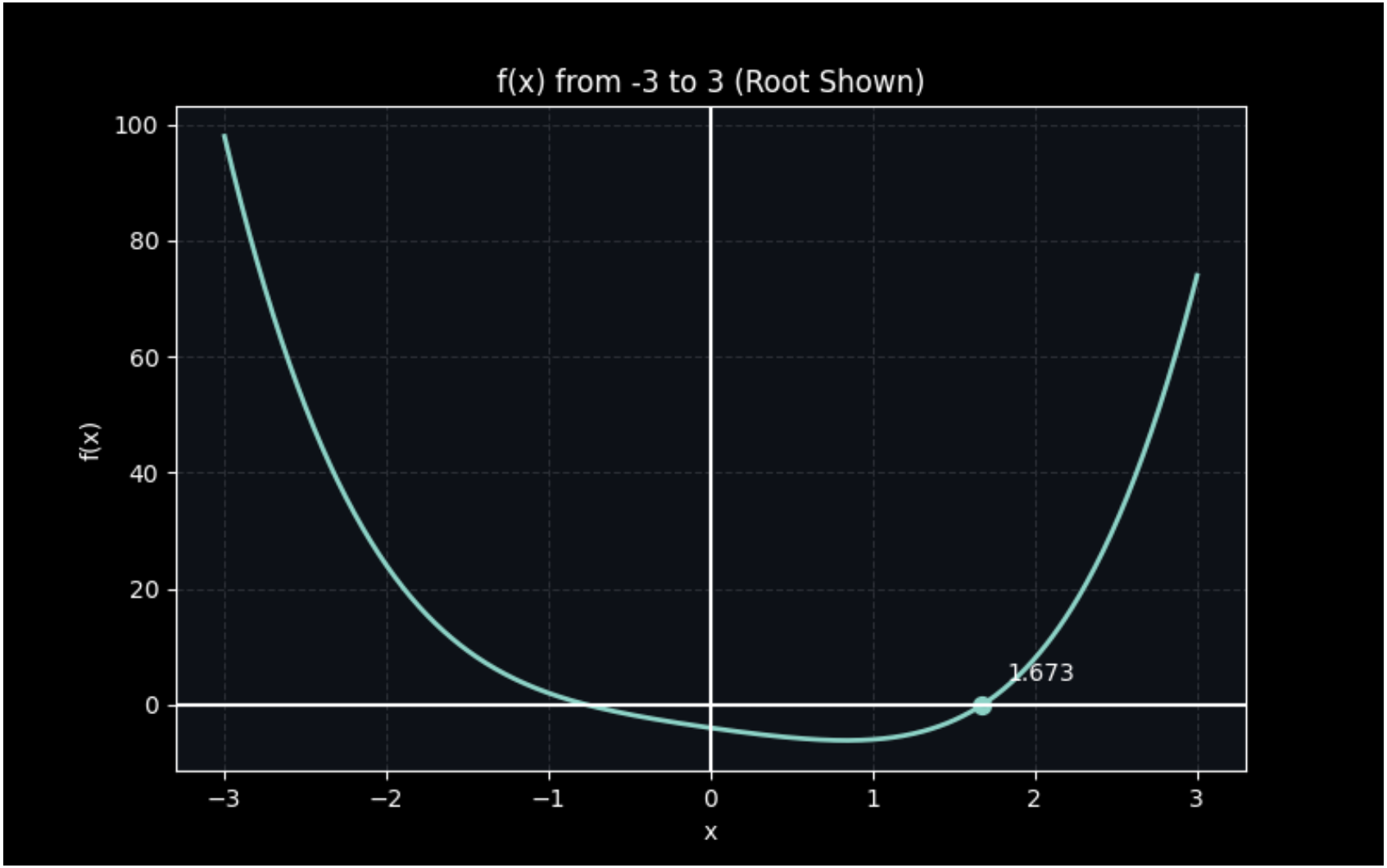


Zoomed view showing tangent intersection clearly at $x = \frac{7}{4}$

Python Code: Full Graph

```
root = 1.6762517

fig3, ax3 = plt.subplots(figsize=(8,5))

x_full = np.linspace(-3, 3, 600)

ax3.plot(x_full, f(x_full))

ax3.axhline(0)
ax3.axvline(0)

ax3.scatter(root, 0)
ax3.annotate("1.673",
 (root, 0),
 textcoords="offset points",
 xytext=(10, 10), ha='left')

plt.show()
```

Graph of $f(x)$



Function plotted from -3 to 3, root at $x \approx 1.673$

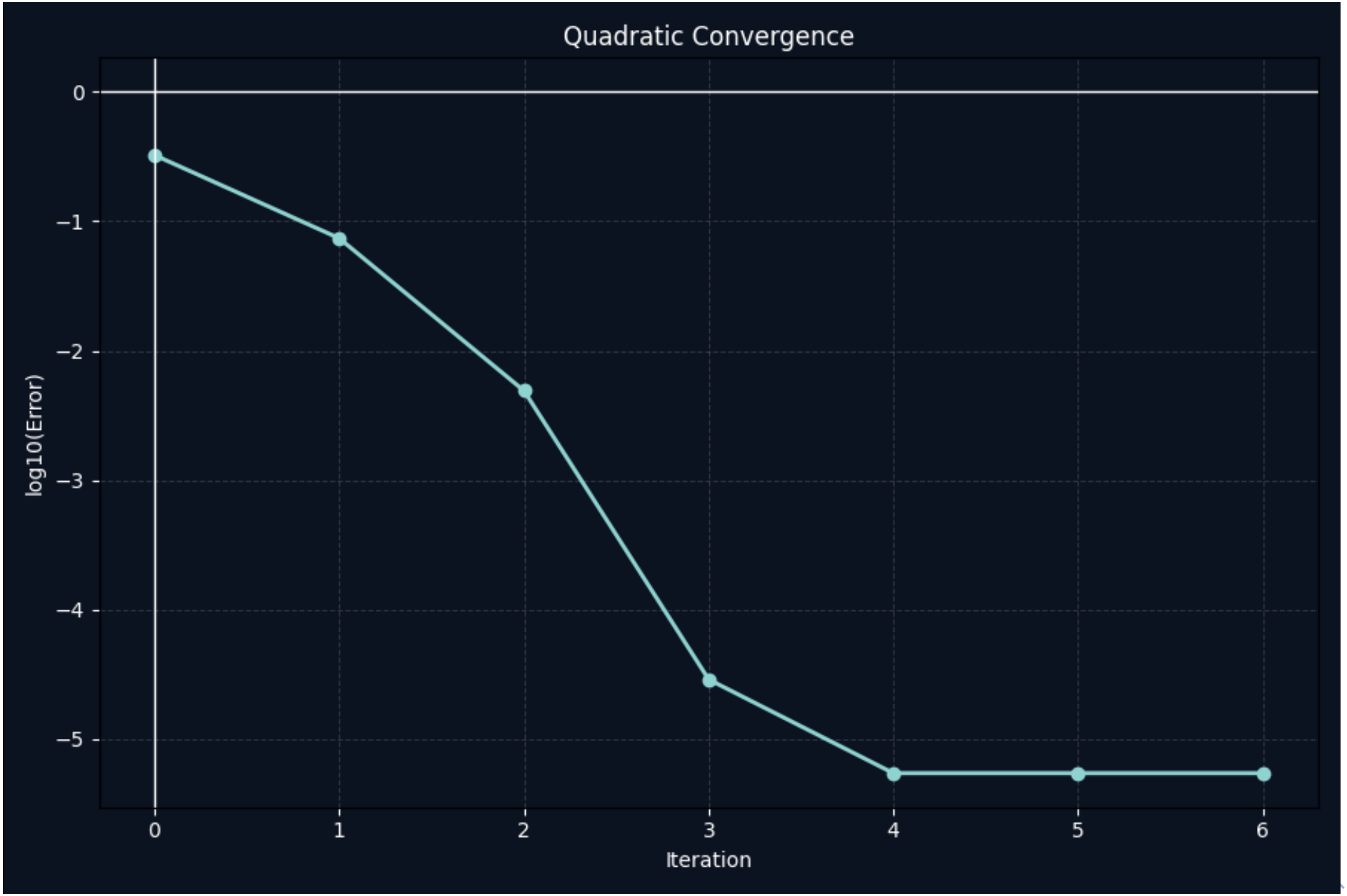
Python Code: Convergence Plot

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
   return x**4 + x**2 - 4*x - 4

def df(x):
   return 4*x**3 + 2*x - 4

true_root = 1.676251739196307

x = 2.0
errors = []
iterations = []

for i in range(7):
   errors.append(abs(x - true_root))
   iterations.append(i)
   x = x - f(x)/df(x)
```

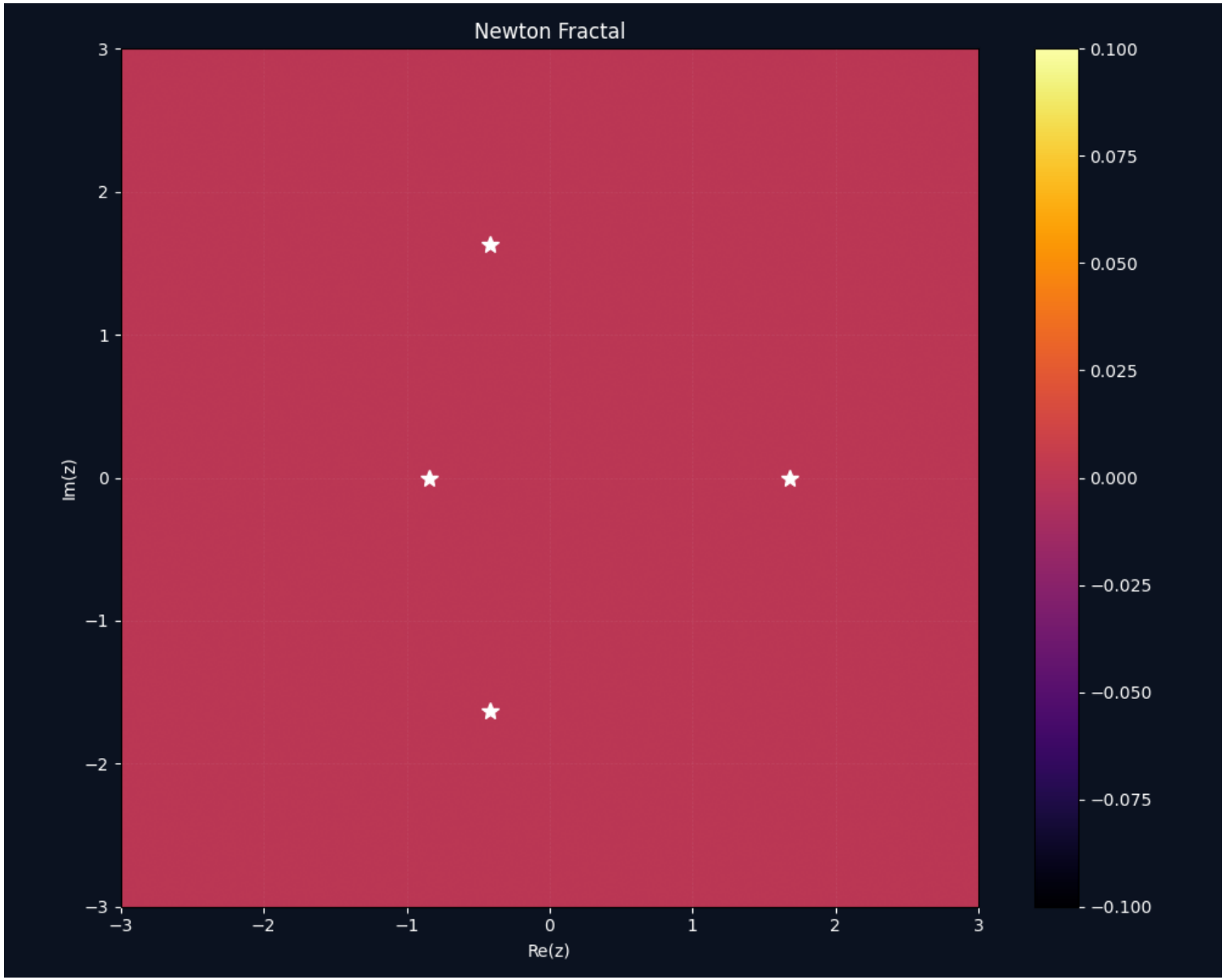
Python Code: Convergence Plot

```
fig, ax = plt.subplots()

ax.plot(iterations, np.log10(errors))
ax.grid(True)

plt.show()
```

Convergence Plot



Python Code: Newton Fractal

```
import numpy as np
import matplotlib.pyplot as plt

def f(z):
   return z**4 + z**2 - 4*z - 4

def df(z):
   return 4*z**3 + 2*z - 4

X, Y = np.meshgrid(np.linspace(-3,3,800),
              np.linspace(-3,3,800))
Z = X + 1j*Y

for _ in range(40):
   Z = Z - f(Z)/df(Z)

plt.imshow(Z.real)
plt.show()
```

Newton Fractal